MODULE 02 · DOMAIN 5 - MODEL SELECTION AND OPTIMISATION

# How Claude Generates

Tokens, the context window, sampling, and why the same prompt can give two different answers.

DOMAIN 5 of 8 EXAM WEIGHT 16.8% MODULE 2 of 36

admin@sthithapragna.com

Claude does not look answers up. It predicts the **next token**, again and again.

# What a token is

- **The unit the model reads and writes:** a word, part of a word, a character or a byte.
- **Older models:** about 3.5 English characters per token.
- **Claude 4.7 and later:** a newer tokenizer, about 30% more tokens for the same text.
- **Varies by language and model.** Count exactly with the token counting endpoint, which is free.

> ON THE EXAM
>
> Prices, rate limits and the context window are all counted in tokens, not words or characters.

# Why tokens matter to a developer

| WHERE TOKENS SHOW UP | WHAT IT MEANS FOR YOU |
|---|---|
| Price | Input and output priced per million tokens; output is 5x input on current models |
| Rate limits | Input and output tokens per minute, as well as requests |
| Context window | Up to 1M tokens on current models, input and output together |
| Latency | Every output token takes time to generate |
| max_tokens | A hard cap on output tokens for one response |

ON THE EXAM

Output tokens cost more and take longer. Asking for less output is often the cheapest optimisation there is.

# One response, one token at a time

01
### Read
The whole request: system, messages, tools, documents.

02
### Score
A probability for every possible next token.

03
### Sample
Pick one token from those odds.

04
### Append
Add it to the text so far.

05
### Repeat
Until end of turn, a tool call, a stop sequence or max_tokens.

THE TAKEAWAY

Generation is a loop of predictions. Nothing in it checks facts against the world.

# What Claude knows versus what Claude sees

TRAINING

## What it learned

- Pretrained to predict text
- Refined with human feedback
- Fixed at a knowledge cutoff
- Your prompt cannot change it

CONTEXT

## What it sees now

- Your system prompt
- The conversation so far
- Tool results and documents
- Fully under your control

THE TIEBREAKER

To give Claude knowledge for this task, change the context. Training a custom model is almost never the exam's answer.

# Everything in the request counts

**Documents**
PDF pages and text

**System prompt**
Role and standing rules

**This turn's output**
Thinking included

**Tool definitions**
Names, descriptions, schemas

**Context window**
Up to 1M tokens
128K output per request
Input plus output
*Working memory for one request*

**Conversation history**
Every earlier turn, resent

**Images**
Photos and scans

**Tool results**
Often the biggest item

**Earlier thinking**
Kept on current models

THE TAKEAWAY

If it is in the request, or Claude writes it, it uses the window.

# When the window fills up

- **Input alone too large:** the API returns a 400 error, "prompt is too long".
- **Input plus max_tokens too large:** accepted on Claude 4.5 and later; generation stops with **model_context_window_exceeded**.
- **Long before that, quality slips:** "As token count grows, accuracy and recall degrade" - context rot.
- **So curate.** Send what this request needs, not everything you have.

> ON THE EXAM
>
> A bigger window is capacity, not a strategy. What you put in matters as much as how much fits.

# The API remembers nothing

```
1  history = [
2      {"role": "user", "content": "Can I keep a cat?"},
3      {"role": "assistant", "content": "Your agreement allows pets with written consent."},
4      {"role": "user", "content": "How do I get consent?"},
5  ]
6  reply = client.messages.create(
7      model="claude-sonnet-5-5", max_tokens=1024,
8      system=SYSTEM_PROMPT, messages=history,
9  )
10 history.append({"role": "assistant", "content": reply.content})
```

1-5 Every earlier turn is sent again, in order.

10 Your code stores the reply and sends it back next time.

THE TAKEAWAY

Statelessness is why long chats cost more every turn, and why session design is your job, not the API's.

# Sampling: picking from the odds

- **The model gives odds for every possible next token.** Sampling chooses one.
- **temperature** - how much randomness: lower favours the most likely token, higher spreads the choice.
- **top_p** - nucleus sampling: keep the smallest set of tokens whose probabilities add up to p.
- **top_k** - keep only the K most likely tokens, cutting the long tail.

> ON THE EXAM
>
> These are fundamentals the exam names. Know what each one does, even where you cannot set it.

# Sampling settings on current models

- **Deprecated:** "Models released after Claude Opus 4.6 do not support setting temperature."
- **temperature** 1.0 is accepted for compatibility; any other value returns a 400 error.
- **top_p** accepted only at 0.99 or above; **top_k** rejected outright.
- **Even at temperature 0**, "the results will not be fully deterministic" - on any model.

> THE RULE
>
> Answer with the concept. Never choose "lower the temperature" as the fix for wrong format, missing facts or a policy breach.

Two runs of the same prompt can differ.
That is a property, not a bug.

# Managing variation by design

| YOU NEED | USE |
| --- | --- |
| An exact format | Structured outputs, then validate in code |
| The same labels every time | A fixed list of categories in the schema, plus examples |
| A consistent style | Examples of the output you want |
| Facts that stay put | Ground answers in supplied documents, with citations |
| Trustworthy test results | Several runs per test case, not one |
| Logic that must never vary | Code, not the model |

THE RULE

Variation is managed with validation, examples and code. That is rule nine: variation is a property.

# Variation, or a bug in your code?

EXPECTED VARIATION

## Same request, different words

- Answer rephrased between runs
- Prompt text echoed one time, paraphrased the next
- Different order, same facts
- Requests identical in the trace

PROBABLY YOUR CODE

## Different requests than you think

- History missing or duplicated
- Different model ID per environment
- Truncated at max_tokens
- Prompt built from stale config

THE TIEBREAKER

Compare the full requests first. Identical requests mean variation; different requests mean a bug.

# Hallucination is fluent prediction

- **The model predicts plausible text.** A plausible citation, figure or clause can be invented.
- **Confidence is not evidence.** A fluent, certain answer can still be wrong.
- **Reduce it:** supply the documents, allow "I don't know", ask for quotes first, check citations.
- **Never fully eliminated**, so check what matters in code or by a person.

> ON THE EXAM
>
> Skepticism toward confident output is an exam objective. Verify anything a decision depends on.

# Copperlet: "just lower the temperature"

- **The symptom:** repair triage labels the same kind of leak "plumbing" one day and "heating" the next.
- **The proposal:** a developer suggests lowering the temperature on the triage model.
- **Problem one:** the current model rejects any temperature but the default.
- **Problem two:** it is the wrong layer. Nothing has been diagnosed yet.
- **The real path:** read the traces, then a fixed category list and examples. Modules 07 and 30.

> COMING BACK TO IT
>
> Evidence first, then a fix at the right layer. We settle this one properly in module 30.

A team estimated the monthly cost of a summary feature by counting words in a sample of documents and pricing them as if one word were one token.

After moving the feature from an older model to Claude Sonnet 5.5, the real bill is well above the estimate, although document volume matches the forecast.

## What best explains the gap, and what should the team do?

A Claude bills by characters, so the team should multiply its word count by average word length

B Billing is per token, and newer models produce more tokens for the same text, so count tokens on the target model

C Output tokens are free, so the gap must come from input documents being longer than expected

D Every current model adds a long-context surcharge, so the team should split documents into smaller requests

◆ PAUSE AND DECIDE

## What best explains the gap, and what should the team do?

A Claude bills by characters, so the team should multiply its word count by average word length

*Billing is per token, not per character.*

B Billing is per token, and newer models produce more tokens for the same text, so count tokens on the target model

Claude 4.7 and later tokenize the same text into about 30% more tokens. Count, do not guess.

C Output tokens are free, so the gap must come from input documents being longer than expected

*Output is billed, and at five times the input price.*

D Every current model adds a long-context surcharge, so the team should split documents into smaller requests

*No long-context premium on current models, except Haiku 5.5 above 100K-token prompts.*

THE SIGNAL

Two errors stacked: words are not tokens, and the tokenizer changed between models. The fix is to measure.

A developer builds a chat feature on the Messages API. Each time a user sends a message, the code sends that single message with the system prompt.

Users complain that the assistant forgets what they said one message earlier.

## What is the cause?

A The model's memory resets after a short idle period, so replies must arrive faster

B Prompt caching is off, so the API cannot store the conversation between requests

C The API is stateless, so the code must send the earlier turns in the messages list

D The context window is too small, so the feature needs a model with a larger window

◆ PAUSE AND DECIDE

## What is the cause?

A The model's memory resets after a short idle period, so replies must arrive faster

*There is no memory to reset. Nothing persists between requests at all.*

B Prompt caching is off, so the API cannot store the conversation between requests

*Caching reuses a repeated prefix you send. It never stores a conversation for you.*

C The API is stateless, so the code must send the earlier turns in the messages list

Each request is all Claude sees. History travels only if you send it.

D The context window is too small, so the feature needs a model with a larger window

*One earlier message is nowhere near the window. It was never sent.*

THE SIGNAL

"Sends that single message" is the signal. If the history is not in the request, Claude cannot see it.

> A product manager asks the same question of an internal assistant twice and receives two differently worded answers. Both are correct.
>
> She worries the application is broken, and asks the developer to explain.

**Which two statements should the developer make? Choose two.**

A Identical requests can produce different wording, even at a temperature of zero

B Different wording proves the application built a different request each time

C Setting the temperature to zero on current models guarantees identical answers

D Pinning a model snapshot removes all variation between runs

E Structured outputs, examples and validation keep variation within limits the team chooses

◆ PAUSE AND DECIDE

## Which two statements should the developer make? Choose two.

**A** Identical requests can produce different wording, even at a temperature of zero

Anthropic says exactly this. Sampling makes variation normal.

**B** Different wording proves the application built a different request each time

*It proves nothing. Identical requests also vary. Check the trace before claiming a bug.*

**C** Setting the temperature to zero on current models guarantees identical answers

*Never guaranteed, and current models reject non-default temperatures.*

**D** Pinning a model snapshot removes all variation between runs

*Pinning stops model changes, not sampling variation.*

**E** Structured outputs, examples and validation keep variation within limits the team chooses

This is how applications manage a property they cannot remove.

THE SIGNAL

Explain the property, then explain the controls. The exam likes answers that do both.

> Copperlet's document reader sends a 60-page tenancy agreement, 40 inventory photos and the text of 12 earlier chats in one request. It all fits comfortably inside the context window.
>
> Accuracy on questions about specific clauses has dropped compared with requests that carried only the agreement.

## What is the most likely explanation?

A More tokens in context lowers accuracy and recall, so each request should carry only what the question needs

B The window overflowed, so the API silently dropped the earliest pages of the agreement

C Photos and PDF pages cannot be read together in the same request

D The default temperature is too high for long documents and should be lowered

◆ PAUSE AND DECIDE

## What is the most likely explanation?

**A** More tokens in context lowers accuracy and recall, so each request should carry only what the question needs

Context rot. Fitting is not the same as helping.

**B** The window overflowed, so the API silently dropped the earliest pages of the agreement

*It fits. And an oversized input is rejected with an error, never trimmed silently.*

**C** Photos and PDF pages cannot be read together in the same request

*They can. Up to 600 images or PDF pages in one request on current models.*

**D** The default temperature is too high for long documents and should be lowered

*Wrong layer, and not a setting current models accept.*

THE SIGNAL

"Fits comfortably" rules out overflow. Falling accuracy with more context is context rot.

A support tool asks a current Claude model for a short case summary with max_tokens set to 300. Thinking is left at the model's default.

Some summaries stop mid-sentence, and their responses show a stop reason of max_tokens.

## What should the developer do?

A Lower the temperature so the model chooses shorter words

B Move to the smallest model tier, because it writes faster

C Add an instruction asking Claude to always finish its sentences

D Raise max_tokens, since it caps thinking too, and set the summary length in the prompt

◆ PAUSE AND DECIDE

## What should the developer do?

A Lower the temperature so the model chooses shorter words

*Sampling does not control length, and current models reject the change.*

B Move to the smallest model tier, because it writes faster

*Speed is not the problem. The cap is hit regardless of speed.*

C Add an instruction asking Claude to always finish its sentences

*Claude cannot finish a sentence the cap has already cut off.*

D Raise max_tokens, since it caps thinking too, and set the summary length in the prompt

Thinking is on by default and counts against the cap. Control length with the prompt instead.

THE SIGNAL

Stop reason max_tokens means the cap cut the answer. On current models the cap covers thinking as well.

> A marketing team wants more varied taglines from Claude Haiku 5.5. A developer adds a temperature of 0.9 to the request to make the output more creative.
>
> Every request now fails with a 400 error.

## What should the developer do?

A. Replace temperature with a top_k value, which the model still accepts

B. Remove the setting and ask for variety in the prompt, such as ten distinct taglines in different styles

C. Move the feature permanently to an older model that still accepts temperature

D. Retry the request with backoff until the API accepts the temperature

◆ PAUSE AND DECIDE

## What should the developer do?

A. Replace temperature with a top_k value, which the model still accepts

*top_k is rejected outright on current models.*

B. Remove the setting and ask for variety in the prompt, such as ten distinct taglines in different styles

Variety is a requirement. State it in the prompt, where current models act on it.

C. Move the feature permanently to an older model that still accepts temperature

*Trades the current model's quality for a dial, and older models retire.*

D. Retry the request with backoff until the API accepts the temperature

*A 400 is a client error. Retrying the same request fails the same way.*

THE SIGNAL

A 400 means the request itself is wrong. Change the request, and say what you want in words.

# Module 2 in five lines

- **Tokens are the unit** for price, rate limits and the window. Count them; newer models produce more.
- **Everything in the request counts,** output and thinking included. More context can mean less accuracy.
- **The API is stateless.** History travels only if your code sends it.
- **Sampling is a concept;** current models reject non-default settings. Variation is managed, not removed.
- **Hallucination is fluent prediction.** Ground answers and verify what matters.

COMING UP

Next: the Messages API in detail - requests, roles, content blocks, stop reasons and vision.